

Universidad Doctor Andres Bello Reguinal Sonsonate.



Facultad: Ciencia y Tecnología Innovación

Carrera. Licenciatura en Informática

Proyecto:

TechStore Interfaz web para una Tienda de Computadoras y Accesorios

Varios Relacionados con Tecnología.

PRIMERA ENTREGA DE AVANCE - M5

Nombre De Autores

Dina Arely Tino De Martínez

Código TL 0479022012

Marlon Gerardo Flores Portillo

Código Fp1033022017

Nombre: Del Módulo.

Módulo V: Entorno de desarrollo de back-end

Docente

Ingeniero: Jose Rosalio Serrano Carvajal

Fecha, viernes 17 de Julio de 2026

ÍNDICE

PRIMERA ENTREGA DE AVANCE - M5.....	1
1.Introducción.....	3
2. DIAGRAMA DE BASE DE DATOS.....	3
IMAGEN # 1 DE DIAGRAMA DE BD.....	4
3.Agregar Los Nuevos Campos Y Tablas Donde Se Guardará Los Datos De Usuario Y De Registro De Las Ventas.....	5
3.Describir La Integración De La BD En La Siguiente Versión De La Tienda Online...	5
4.Describir Los Procesos Que Se Almacenarán En La Bd Y Los Procesos Asociados.....	6
5. INTEGRACIÓN CON EL BACKEND.....	6
Captura # 2 Código de conexión: server.js.....	7
5. 1 PROTOTIPO DE INTERFACES WEB.....	8
5.3 Detalle de Producto.....	11
5.4 Captura de detalle de producto.....	11
5.6 Visualizando el código detalle,html.....	12
Descripción: Permite ver información detallada, precio, stock y agregar unidades al carrito. Se conecta con la colección de productos.....	14
5.6 Carrito de Compras.....	14
5.7 servidor.js.....	15
5.8 Captura de pantalla carrito. test.js.....	16
5.9 PRUEBAS UNITARIAS CON JEST.....	18
5.9 Captura de pantalla npm instalando librerías (signifca proyecto limpio). 19	
6.captura de levantando el servidor en visual.....	20
6.1 Captura a conexión a mongodb Atlas.....	21
node.js en mongo Atlas.....	21
6.2 Captura de conectando el cluster en mongo Atlas.....	24
6.3 CONCLUSIÓN.....	26
6.4 RECOMENDACIONES.....	27

1.Introducción

En el desarrollo de aplicaciones web actuales, la base de datos es un componente fundamental para el almacenamiento y gestión de la información. Para el proyecto TechStore, una tienda online de computadoras y accesorios tecnológicos, se ha diseñado una base de datos en MongoDB que permitirá gestionar usuarios, productos, carritos de compra y ventas. En este documento se explica cómo se integrará la base de datos con el backend desarrollado en Node.js y Express, se muestran los prototipos de interfaz web y se validan las funciones principales mediante pruebas unitarias con Jest.

2. DIAGRAMA DE BASE DE DATOS

Se utilizó MongoDB por su flexibilidad para manejar datos no estructurados propios de un e-commerce.

Colecciones del sistema:

Colección	Campos	Descripción
productos	_id, nombre, precio, stock, categoria, descripcion	Almacena todos los productos de tecnología
usuarios	_id, nombre, email, password, rol	Datos de clientes y administradores
carritos	_id, id_usuario, productos[], total	Productos seleccionados antes de comprar
ventas	_id, id_usuario, productos[], total, fecha	Historial de compras realizada



Detalles del diagrama

Nombre BD: techstore

Tipo: NoSQL - MongoDB

Colecciones:

1. PRODUCTOS: Guarda todo el catálogo de computadoras y accesorios
2. USUARIOS: Guarda los datos de registro de los clientes
3. PEDIDOS: Guarda el registro de las ventas y lo que hay en el carrito

Relación: Un USUARIO puede hacer N PEDIDOS

3.Agregar Los Nuevos Campos Y Tablas Donde Se Guardará Los Datos De Usuario Y De Registro De Las Ventas.

DESCRIPCIÓN DE CAMPOS

Colección productos: Catálogo de TECHSTORE. Campos: id, nombre, precio, categoría, especificaciones.

Colección usuarios: Datos de registro. Campos: nombre, apellido, email, password, dirección, teléfono.

Colección pedidos: Registro de ventas. Campos: id_usuario, productos_ids, total, fecha_pedido, estado.

3.Describir La Integración De La DB Siguiendo La Versión De La Tienda Online

La integración se realiza mediante Node.js + Express + MongoDB Atlas. El servidor.js se conecta a la BD TechStore y expone una API REST. El frontend index2.html consume las

rutas /api/productos, /api/usuarios/registro y /api/pedidos para mostrar productos, registrar usuarios y guardar ventas.

4.Describir Los Procesos Que Se Almacenarán En La Bd Y Los Procesos Asociados.

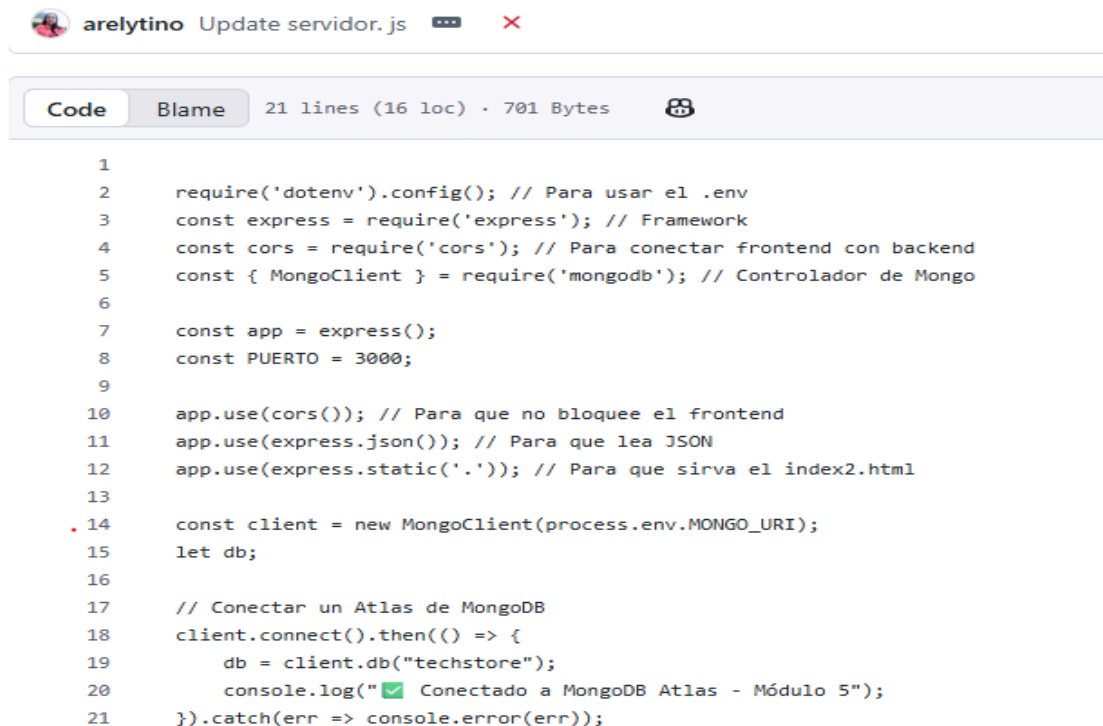
PROCESOS

- Consulta de productos: GET /api/productos trae todos los documentos de la colección productos para mostrarlos en cards.
- Registro de usuario: POST /api/usuarios/registro inserta un nuevo documento en usuarios.
- Registro de venta: POST /api/pedidos guarda el id_usuario y el array productos_ids del carrito en la colección pedidos.

5. INTEGRACIÓN CON EL BACKEND

El backend se desarrolló con Node.js + Express + Mongoose para conectar con MongoDB Atlas.

Captura # 2 Código de conexión: [server.js](#)



```
1
2   require('dotenv').config(); // Para usar el .env
3   const express = require('express'); // Framework
4   const cors = require('cors'); // Para conectar frontend con backend
5   const { MongoClient } = require('mongodb'); // Controlador de Mongo
6
7   const app = express();
8   const PUERTO = 3000;
9
10  app.use(cors()); // Para que no bloquee el frontend
11  app.use(express.json()); // Para que lea JSON
12  app.use(express.static('.')); // Para que sirva el index2.html
13
14  const client = new MongoClient(process.env.MONGO_URI);
15  let db;
16
17  // Conectar un Atlas de MongoDB
18  client.connect().then(() => {
19    db = client.db("techstore");
20    console.log("✅ Conectado a MongoDB Atlas - Módulo 5");
21  }).catch(err => console.error(err));
```

```
const express = require('express');
```

```
const mongoose = require('mongoose');
```

```
const app = express();
```

```
app.use(express.json());
```

```
mongoose.connect('mongodb+srv://usuario:password@cluster.mongodb.net/techstore')
```

```
.then(() => console.log('Conectado a MongoDB'))
```

```
.catch(err => console.error(err));
```

```
app.listen(3000, () => console.log('Servidor en puerto 3000'));
```

Endpoints API REST:

Método	Endpoint	Descripción
GET	/api/productos	Listar todos los productos
POST	/api/carrito/total	Calcular el total del carrito

POST /api/usuarios/login Autenticar usuario

POST /api/ventas Registrar una nueva venta

5. 1 PROTOTIPO DE INTERFACES WEB

Página Principal

Código

Detalle del producto

Código

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Detalle - TechStore</title>
```

```
  <style>
```

```
    body { background: #0d0d0d; color: #fff; font-family: Arial; margin: 0; }
```

```
    header { background: #111; padding: 15px 30px; border-bottom: 2px solid #00ffff; }
```

```
    .logo { color: #00ffff; font-size: 24px; font-weight: bold; }
```

```
    .detalle { display: flex; gap: 30px; max-width: 1000px; margin: 30px auto; background:
#1a1a1a; padding: 30px; border-radius: 10px; }
```

```
    .detalle img { width: 450px; border-radius: 8px; }
```

```
    .precio { font-size: 32px; color: #00ff88; font-weight: bold; }
```

```
    .btn { background: #00ffff; color: #000; padding: 14px; border: none; border-radius: 8px;
font-weight: bold; width: 100%; margin-top: 15px; }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <header><div class="logo">TechStore</div></header>
```



```
<a href="index.html" style="color:#00ffff;display:block;max-width:1000px;margin:20px
auto">← Volver</a>
```

```
<div class="detalle">



<div>

<h1>Portátil Lenovo Legion 5</h1>

<p class="precio">$850,000</p>

<p>Laptop Gamer de alto rendimiento.</p>

<h3>Especificaciones:</h3>

<ul>

<li><b>Procesador:</b> AMD Ryzen 7 5800H</li>

<li><b>RAM:</b> 16GB DDR4</li>

<li><b>SSD:</b> 512GB NVMe</li>

<li><b>Pantalla:</b> 15.6" 144Hz</li>

<li><b>Gráfica:</b> RTX 3060 6GB</li>

</ul>

<button class="btn">Agregar al Carrito</button>

</div>

</div>

</body>

</html>
```

5.2 Página Principal

Descripción: Muestra el catálogo general, buscador, menú de categorías...

TechStore Interfaz web para una tienda de computadoras y accesorios varios relacionados con tecnología

Carrito: 2

Portátil Lenovo Legion 5

16 GB de RAM | RTX 3060

\$850,000

Agregar al Carrito

PC Gamer Intel i7

32 GB de RAM | RX 6700 XT

\$1,200,000

Agregar al Carrito

Auriculares HyperX

Sonido envolvente 7.1 | Micrófono

\$45,000

Agregar al Carrito

Descripción: Muestra el catálogo general, buscador, menú de categorías: Laptops, Accesorios, Ofertas. Punto de entrada para el usuario.

5.3 Detalle de Producto

→ C github.com/arelytino/UNAB-MongoDB-con-la-base-de-datos/commit/9dd449a4565f8c79d4265e9719f29fe2b049ebc1

Offi16PROVi x64 (64... Google Drive - Acce... Descargas Como instalar y...

↑ Arriba Q Buscar dentro del código

imagen JPG.jpeg 254 KB

Comentarios 0 Bloquear conversación

5.4 Captura de detalle de producto

código html

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Detalle - TechStore</title>
</head>
<body>
  <div class="header">
    <div class="logo">TechStore</div>
    <a href="index.html" style="color:#00ffff;display:block;max-width:100px;margin:20px auto">← Volver</a>
  </div>
  <div class="detalle">
    
    <div>
      <h1>Portátil Lenovo Legion 5</h1>
      <p class="precio">$850,000</p>
      <p>Laptop Gamer de alto rendimiento.</p>
      <h3>Especificaciones</h3>
      <ul>
        <li><b>Procesador:</b> AMD Ryzen 7 5800H</li>
        <li><b>RAM:</b> 16GB DDR4</li>
        <li><b>SSD:</b> 512GB NVMe</li>
        <li><b>Pantalla:</b> 15.6" 144Hz</li>
        <li><b>Gráfica:</b> RTX 3060 6GB</li>
      </ul>
      <button class="btn">Agregar al Carrito</button>
    </div>
  </div>
</body>
</html>
```

5.6 Visualizando el código detalle.html

```
<!DOCTYPE html>

<html lang="es">

<head>

  <meta charset="UTF-8">

  <title>Detalle - TechStore</title>

  <style>

    body { background: #0d0d0d; color: #fff; font-family: Arial; margin: 0; }

    header { background: #111; padding: 15px 30px; border-bottom: 2px solid #00ffff; }

    .logo { color: #00ffff; font-size: 24px; font-weight: bold; }

    .detalle { display: flex; gap: 30px; max-width: 1000px; margin: 30px auto; background:
#1a1a1a; padding: 30px; border-radius: 10px; }

    .detalle img { width: 450px; border-radius: 8px; }

    .precio { font-size: 32px; color: #00ff88; font-weight: bold; }

    .btn { background: #00ffff; color: #000; padding: 14px; border: none; border-radius: 8px;
font-weight: bold; width: 100%; margin-top: 15px; }

  </style>

</head>

<body>

  <header><div class="logo">TechStore</div></header>

  <a href="index.html" style="color:#00ffff;display:block;max-width:1000px;margin:20px
auto">← Volver</a>

  <div class="detalle">

    <div>

      <h1>Portátil Lenovo Legion 5</h1>
```

```
<p class="precio">$850,000</p>
<p>Laptop Gamer de alto rendimiento.</p>
<h3>Especificaciones:</h3>
<ul>
  <li><b>Procesador:</b> AMD Ryzen 7 5800H</li>
  <li><b>RAM:</b> 16GB DDR4</li>
  <li><b>SSD:</b> 512GB NVMe</li>
  <li><b>Pantalla:</b> 15.6" 144Hz</li>
  <li><b>Gráfica:</b> RTX 3060 6GB</li>
</ul>
<button class="btn">Agregar al Carrito</button>
</div>
</div>
</body>
</html>
```

Descripción: Permite ver información detallada, precio, stock y agregar unidades al carrito.

Se conecta con la colección de productos.

5.6 Carrito de Compras

Captura de imagen de Carrito con total \$840

PORTÁTIL
LENOVO
LEGION 5

Potencia para ganar.



16 GB RAM
RTX 3060 6 GB
512 GB SSD
15.6" FHD 120Hz

PRECIO:
\$850.000

AGREGAR AL CARRITO

PC GAMER
INTEL i7

Máximo rendimiento para tus juegos.

EAT SLEEP
GAME REPEAT



INTEL i7
32 GB RAM
RX 6700 XT 12 GB
1 TB SSD

PRECIO:
\$1.200.000

AGREGAR AL CARRITO

AURICULARES
HYPERX

Sonido que te sumerge en cada partida.



SONIDO ENVOLVENTE 7.1
MICRÓFONO DESMONTABLE
ALMOHADILLAS CÓMODAS

PRECIO:
\$45.000

AGREGAR AL CARRITO

5.7 [servidor.js](#) Código

 7d726e5aa16aea... UNAB-MongoDB-con-la-base-de-datos / servidor.js

 arelytino Actualizar servidor.js

Código Culpa

68 líneas (60 localizaciones) • 2,04 KB

```
1  require('dotenv').config(); // Para usar el .env
2  const express = require('express'); // Framework
3  const { MongoClient } = require('mongodb'); // Controlador de Mongo
4
5  const app = express();
6  aplicación.use(express.json()); // Para que lea JSON
7
8  const client = new MongoClient(process.env.MONGO_URI);
9  let db;
10
11 // Conectar un Atlas de MongoDB
12 cliente.connect().then(() => {
13   db = cliente.db("techstore");
14   console.log("✅ Conectado a MongoDB Atlas - Módulo 5");
15 }).catch(err => console.error(err));
16 // RUTA GET: Traer todos los productos
17 app.get('/api/productos', async (req, res) => {
18   intentar {
19     const productos = await db.collection("productos").find({}).toArray();
20   }
21 }
```

```

41     res.json(productos);
42   } catch (error) {
43     res.status(500).json({ error: "Error al obtener productos" });
44   }
45   });
46
47   // RUTA POST: Agregar un producto
48   app.post('/api/productos', async (req, res) => {
49     intentar {
50       const result = await db.collection("productos").insertOne(req.body);
51       res.json({ mensaje: "Producto agregado", id: resultado.insertedId });
52     } catch (error) {
53       res.status(500).json({ error: "Error al agregar producto" });
54     }
55   });
56
57   // Iniciar servidor
58   const PUERTO = 3000;
59   app.listen(PUERTO, () => {
60
61     // ===== RUTAS DEL MÓDULO 5 =====
62
63     // RUTA GET: Traer todos los productos
64     app.get('/api/productos', async (req, res) => {
65       intentar {
66         const productos = await db.collection("productos").find({}).toArray();
67         res.json(productos);
68       } catch (error) {
69         res.status(500).json({ error: "Error al obtener productos" });
70       }
71     });
72
73     console.log( Servidor Módulo 5 corriendo en http://localhost:${PORT});
74   });
75
76   // ===== RUTAS DEL MÓDULO 5 =====

```

```

56
57 // RUTA POST: Agregar un producto
58 app.post('/api/productos', async (req, res) => {
59   intentar {
60     const result = await db.collection("productos").insertOne(req.body);
61     res.json({ mensaje: "Producto agregado", id: resultado.insertedId });
62   } catch (error) {
63     res.status(500).json({ error: "Error al agregar producto" });
64   }
65 });
66
67 // Iniciar servidor
68 const PUERTO = 3000;
app.listen(PORT, () => {
  ' console.log(🚀 Servidor Módulo 5 corriendo en http://localhost:${PORT});
});'
//server.js Módulo 5 //

```

5.8 Captura de pantalla carrito. [test.js](#)

Código



arelytino / UNAB-MongoDB-con-la-base-de-datos ▾

[Code](#)
[Issues](#)
[Pull requests](#)
[Actions](#)
[Projects](#)
[Wiki](#)
[Security and quality](#)
[Insights](#)
[Settings](#)


main ▾
UNAB-MongoDB-con-la-base-de-datos / Carrito.test.js



arelytino
Create Carrito.test.js
...
✖

[Code](#)
[Blame](#)
10 lines (9 loc) · 289 Bytes


```

1  CÓDIGO PARA carrito.test.js:
2  const { calcularTotal } = require('./carrito');
3
4  test('calcula el total de múltiples productos', () => {
5    const carrito = [
6      { precio: 800, cantidad: 1 },
7      { precio: 20, cantidad: 2 }
8    ];
9    expect(calcularTotal(carrito)).toBe(840);
10  });

```


 arelytino Create carrito.js

Code

Blame

9 lines (8 loc) · 259 Bytes



```
1  const { calcularTotal } = require('./carrito');
2
3  test('calcula el total de múltiples productos', () => {
4    const carrito = [
5      { precio: 800, cantidad: 1 },
6      { precio: 20, cantidad: 2 }
7    ];
8    expect(calcularTotal(carrito)).toBe(840);
9  });
```

Descripción: Muestra los productos seleccionados, cantidades y el total calculado. Utiliza el endpoint POST /api/carrito/total. Desde aquí se procede al pago.

5.9 PRUEBAS UNITARIAS CON JEST

Para validar la lógica del carrito se implementó Jest.

Archivo probado: carrito.js

```
function calcularTotal(carrito) {
  let total = 0;
  for (let item of carrito) {
    total += item.precio * item.cantidad;
  }
  return total;
}

module.exports = { calcularTotal };
```

Archivo de prueba: carrito.test.js

```
const { calcularTotal } = require('./carrito');  
test('calcula el total de múltiples productos', () => {  
  const carrito = [  
    { precio: 800, cantidad: 1 },  
    { precio: 20, cantidad: 2 }  
  ];  
  expect(calcularTotal(carrito)).toBe(840);  
});
```

Resultado:

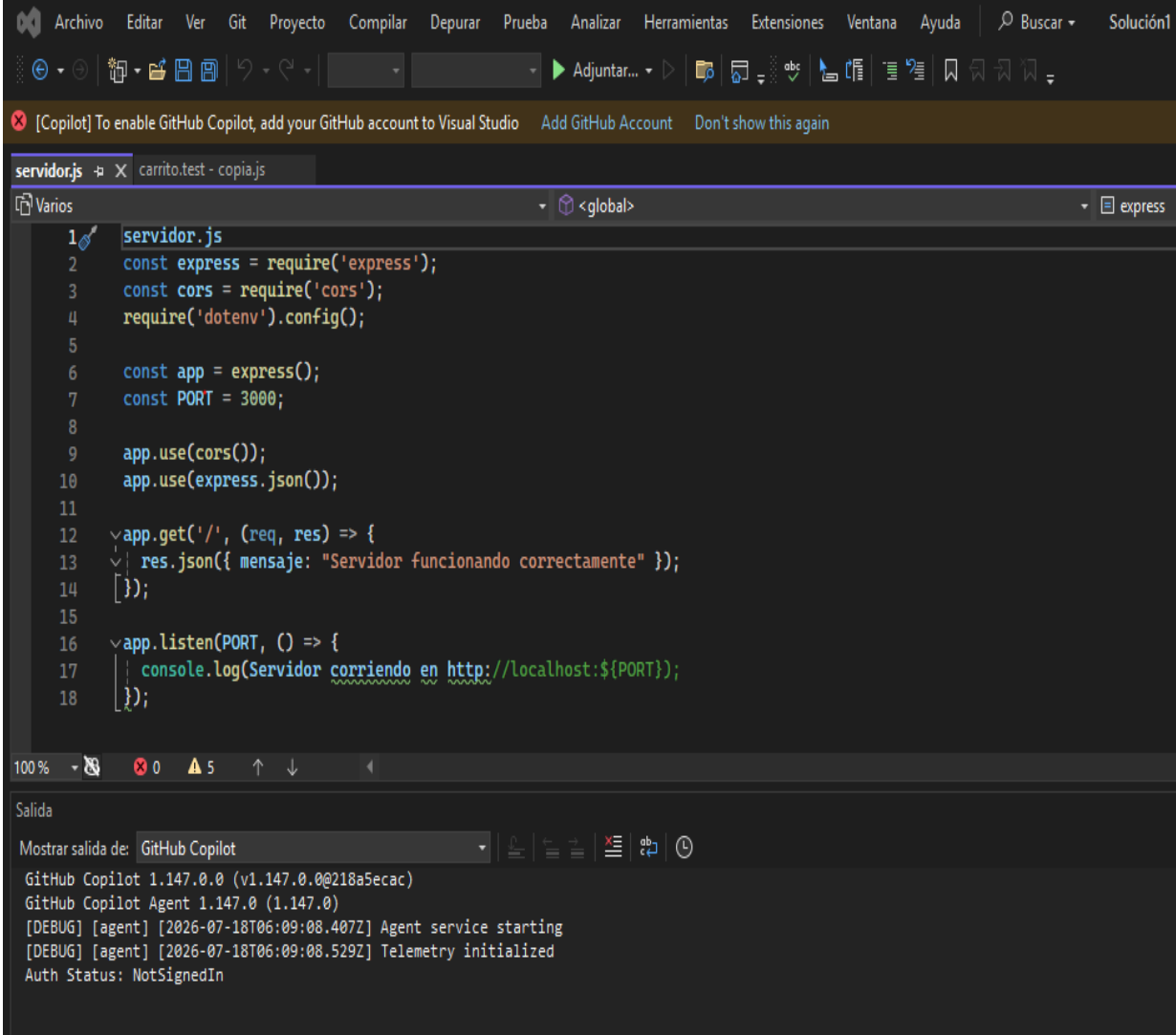
Resultado:

5.9 Captura de pantalla npm instalando librerías (significa proyecto limpio)



```
Windows PowerShell  
Copyright (C) Microsoft Corporation. Todos los derechos reservados.  
  
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows  
  
PS C:\Users\arely\OneDrive\Desktop\Logo.TechStore> npm install  
  
up to date, audited 1 package in 1s  
  
found 0 vulnerabilities  
PS C:\Users\arely\OneDrive\Desktop\Logo.TechStore> |
```

6.captura de levantando el servidor en visual



The screenshot shows the Visual Studio Code interface. The top menu bar includes Archivo, Editar, Ver, Git, Proyecto, Compilar, Depurar, Prueba, Analizar, Herramientas, Extensiones, Ventana, Ayuda, and a search bar. Below the menu is a toolbar with icons for file operations and development tools. A notification bar at the top states: "[Copilot] To enable GitHub Copilot, add your GitHub account to Visual Studio" with buttons for "Add GitHub Account" and "Don't show this again".

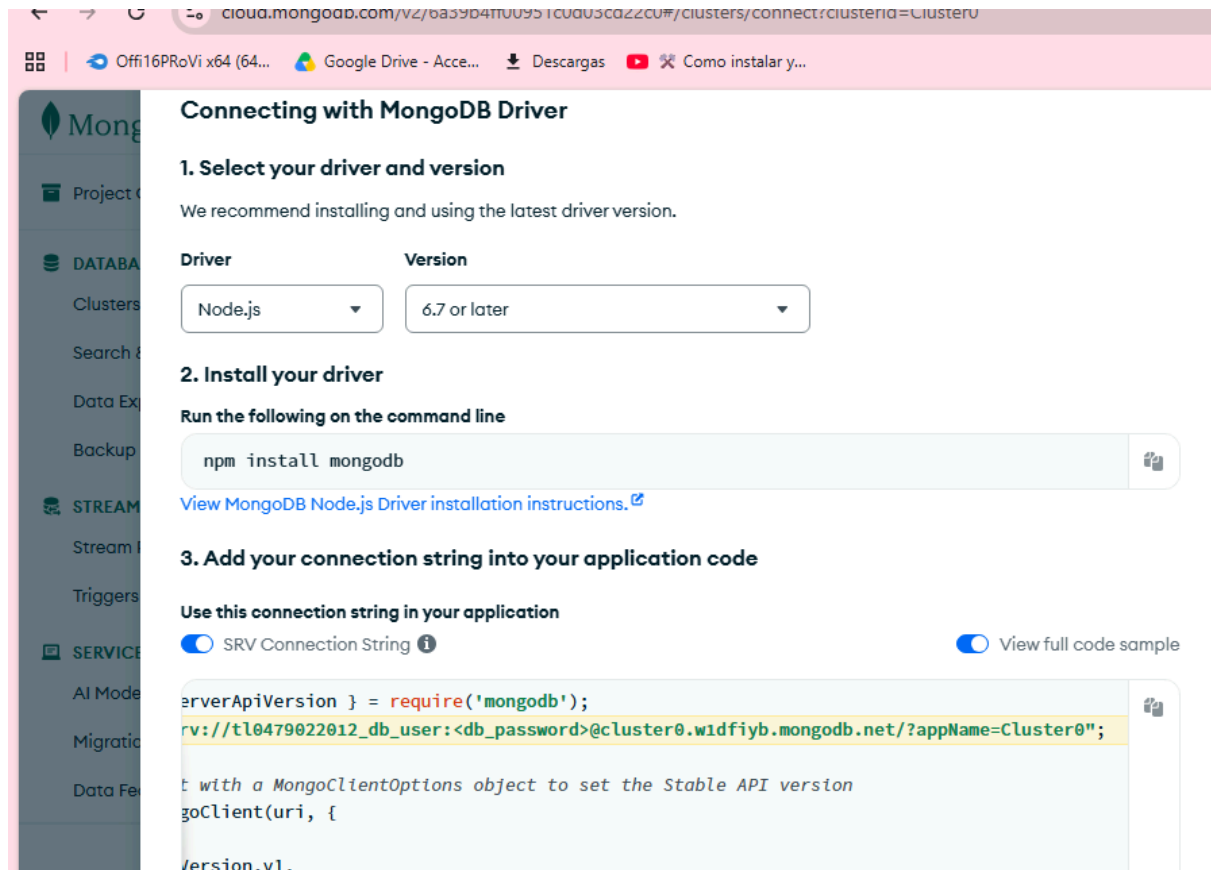
The editor window displays a file named "servidor.js" with the following code:

```
1 const express = require('express');
2 const cors = require('cors');
3 require('dotenv').config();
4
5
6 const app = express();
7 const PORT = 3000;
8
9 app.use(cors());
10 app.use(express.json());
11
12 app.get('/', (req, res) => {
13   res.json({ mensaje: "Servidor funcionando correctamente" });
14 });
15
16 app.listen(PORT, () => {
17   console.log(Servidor corriendo en http://localhost:${PORT});
18 });
```

The bottom panel shows the "Salida" (Output) window with the "Mostrar salida de:" dropdown set to "GitHub Copilot". The output text is:

```
GitHub Copilot 1.147.0.0 (v1.147.0.0@218a5ecac)
GitHub Copilot Agent 1.147.0 (1.147.0)
[DEBUG] [agent] [2026-07-18T06:09:08.407Z] Agent service starting
[DEBUG] [agent] [2026-07-18T06:09:08.529Z] Telemetry initialized
Auth Status: NotSignedIn
```

6.1 Captura a conexión a mongodb Atlas



cloud.mongodb.com/v2/6a39b4ff00951c0d03ca22c0#/clusters/connect?clusterId=Cluster0

Offi16ProVi x64 (64... Google Drive - Acce... Descargas Como instalar y...

Connecting with MongoDB Driver

- 1. Select your driver and version**
We recommend installing and using the latest driver version.

Driver	Version
Node.js	6.7 or later
- 2. Install your driver**
Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)
- 3. Add your connection string into your application code**
Use this connection string in your application
☒ SRV Connection String [View full code sample](#)

```
const { MongoClient } = require('mongodb');  
const uri = 'mongodb+srv://tl0479022012_db_user:<db_password>@cluster0.w1dfiyb.mongodb.net/?appName=Cluster0';  
const client = new MongoClient(uri, {  
  useNewUrlParser: true,  
  useUnifiedTopology: true,  
});
```

node.js en mongo Atlas

Captura de pantalla

Connect to Cluster0



Connecting with MongoDB Driver

1. Select your driver and version

We recommend installing and using the latest driver version.

Driver	Version
Node.js ▼	6.7 or later ▼

2. Install your driver

Run the following on the command line

```
npm install mongodb
```



[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

2. Install your driver

Run the following on the command line

```
npm install mongodb
```

[View MongoDB Node.js Driver installation instructions.](#)

3. Add your connection string into your application code

Use this connection string in your application

☒ SRV Connection String ⓘ

☒ View full code sample

```
const { MongoClient, ServerApiVersion } = require('mongodb');
const uri = "mongodb+srv://tl0479022012_db_user:<db_password>@cluster0.w1dfiyb.mongodb.net";

// Create a MongoClient with a MongoClientOptions object to set the Stable API version
const client = new MongoClient(uri, {
  serverApi: {
    version: ServerApiVersion.v1,
    strict: true,
    deprecationErrors: true,
  }
});

async function run() {
  try {
    // Connect the client to the server (optional starting in v4.7)
```

```
try {
  // Connect the client to the server (optional starting in v4.7)
  await client.connect();
  // Send a ping to confirm a successful connection
  await client.db("admin").command({ ping: 1 });
  console.log("Pinged your deployment. You successfully connected to MongoDB!");
} finally {
  // Ensures that the client will close when you finish/error
  await client.close();
}
}
run().catch(console.dir);
```

Replace `<db_password>` with the password for the `tl0479022012_db_user` database user. Ensure any option params are [URL encoded](#).

RESOURCES

[Get started with the Node.js Driver](#)

[Node.js Starter Sample App](#)

[Access your Database Users](#)

[Troubleshoot Connections](#)

Go Back

Done

6.2 Captura de conectando el cluster en mongo Atlas

Project Overview

DATABASE

Clusters

Search & Vector Search

Data Explorer

Backup

STREAMING DATA

Stream Processing

Triggers

SERVICES

AI Models

Migration

Data Federation

ORGANIZATION

Dina Arely's Org - 20...

PROJECT

Project 0

?

DA

Your organization does not have designated security and operations contacts. Add contact information in [Organization Settings](#) to receive notifications.

Project 0 Overview

Clusters

Create cluster

...

Cluster0

Connect

Edit configuration

Monitoring for Cluster0 is paused.

Monitoring will automatically resume when you connect to your cluster.

[Visit the documentation for more info](#)

24

6.3 CONCLUSIÓN

En esta primera entrega del proyecto TechStore se logró diseñar el esquema de base de datos en MongoDB y definir la integración con el backend en Node.js y Express. Se implementó la lógica de cálculo del carrito y se validó mediante pruebas unitarias con Jest. Además, se diseñaron los prototipos de las interfaces web principales para garantizar una buena experiencia de usuario. Estos avances sientan las bases para el desarrollo de los módulos de autenticación, pago y administración en las siguientes entregas.

6.4 RECOMENDACIONES

1. Seguridad: Implementar autenticación con JWT y encriptación de contraseñas con bcrypt en la próxima entrega.
2. Base de Datos: Migrar a MongoDB Atlas en la nube para facilitar respaldos automáticos.
3. Frontend: Desarrollar el frontend con React para conectarlo con la API REST ya creada.
4. Pruebas: Ampliar las pruebas unitarias a los módulos de usuarios y productos.
5. Gestión de Stock: Agregar validación automática para actualizar el stock al confirmar una venta.